

Reviewer Pack — Minimal Rank of Brauer Groups over Function Fields vs Exponential Depth of Frege Proofs

Entry f10b0d9c9ad2 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-24 16:03:08 UTC

Minimal Rank of Brauer Groups over Function Fields vs Exponential Depth of Frege Proofs

Entry ID: f10b0d9c9ad2 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-24 16:03:08 UTC

1. Conjecture

Field A (mathematical branch): Algebraic Geometry (Brauer Group Theory) **Field B** (complexity object): Complexity Theory: Frege Proof Complexity

Statement:

{'complexity_object': 'Frege proof complexity', 'algebraic_geometry_object': 'Minimal rank of Brauer groups over function fields', 'quantitative_relation': 'For all instances with n variables, $E[\text{rank}(\text{Brauer}(G_n))] = \Theta(2^{n/3})$ for Frege proofs of depth 3.'}

Rationale (proposer's reasoning):

{'connection_explanation': 'The minimal rank of Brauer groups measures the complexity of certain field extensions. This invariant could potentially reflect the depth of logical inferences required in Frege proofs, as both involve abstract algebraic structures.', 'potential_structure_exposure': 'If the conjecture holds, it suggests a deep connection between number theory and computational complexity that may expose new insights into the nature of efficient computation.'}

Taxonomy category: GEOMETRIC_ALGEBRAIC_COMPLEXITY (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 2df4e060923d13a3

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if the Spearman's rank correlation coefficient between the minimal rank of Brauer groups over function fields and the exponential depth of Frege proofs exceeds 0.7 for at least 30 seeds, with a mean absolute difference between the expected rank and actual rank less than or equal to 2.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	1.00	HITS	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	UNCERTAIN	1.00	UNCERTAIN	HITS
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 3 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - Minimal rank Brauer groups function fields AND Frege proof complexity - Algebraic Geometry Brauer group theory AND Complexity Theory Frege proof depth - Exponential depth Frege proofs AND algebraic geometry Minimal rank Brauer groups

Top relevant hits considered: - [<http://arxiv.org/abs/math/0408006v2>] Some remarks on Brauer groups of K3 surfaces - [<http://arxiv.org/abs/1405.2700v1>] Zero Excess and Minimal Length in Finite Coxeter Groups - [<http://arxiv.org/abs/1306.6538v4>] The Brauer group is not a derived invariant

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=124, elapsed=240.3s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_3cnf(n):
        clauses = []
        for _ in range(2**n):
            clause = [random.choice([-1, 1]) * (i + 1) for i in range(n)]
            if any(clause.count(x) > 1 or clause.count(-x) > 1 for x in set(clause)):
                continue
            clauses.append(clause)
        return clauses

    def calculate_brauer_rank(clauses):
        # Placeholder function to simulate Brauer rank calculation
        return len(clauses) / n

    def measure_frege_complexity(clauses):
        # Placeholder function to simulate Frege complexity measurement
        return 2**n

    n = random.randint(5, 40)
    clauses = generate_3cnf(n)
```

```

brauer_rank = calculate_brauer_rank(clauses)
frege_complexity = measure_frege_complexity(clauses)

metric_value = abs(brauer_rank - (2**n / 3))
conjecture_holds = metric_value <= 2
counterexample = "" if conjecture_holds else f"brauer_rank={brauer_rank}, frege_complexity={frege_complexity}"

return {
    "metric_name": "Brauer Rank vs Frege Complexity",
    "metric_value": metric_value,
    "instances_tested": 1,
    "conjecture_holds": conjecture_holds,
    "counterexample": counterexample
}

if __name__ == "__main__":
    if len(sys.argv) > 1:
        seeds = [int(s) for s in sys.argv[1:]]
    else:
        primes = [2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47, 53, 59, 61, 67, 71, 73]
        seeds = random.sample(primes, min(30, len(primes)))

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_metric_value = sum(r["metric_value"] for r in results) / len(results)
    std_metric_value = math.sqrt(sum((r["metric_value"] - mean_metric_value)**2 for r in results) / len(results))
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_metric_value} std={std_metric_value} support_fraction={support_fraction}")
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_metric_value} std={std_metric_value} support_fraction={support_fraction}")
    else:
        first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample={results[0]['counterexample']} first_failing_seed={first_failing_seed}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

TIMEOUT after 240s

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test timed out before producing data, which means it did not complete its execution to evaluate the conjecture. | next: Re-run the test with increased time limits or optimize the code to ensure it completes within the given time frame.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	11639
2	preregistration	ollama_remote	glm4:latest	0	0	5846
3	novelty	ollama_remote	glm4:latest	0	0	4645
4	novelty	ollama_remote	glm4:latest	0	0	5877
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	18486
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10586
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9715
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	8796
9	judge	ollama_remote	glm4:latest	0	0	16646

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 92235 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in research/audit/f10b0d9c9ad2.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/f10b0d9c9ad2.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/f10b0d9c9ad2.tar.gz (if generated)